T.Mubeena
192425295

NATURAL LANGUAGE PROCESSING

1. Design an FSA to Recognize Absolute Date Expressions

A Finite State Automaton (FSA) is a mathematical model used to recognize patterns in a sequence of input symbols. It consists of a finite number of states and transitions. The automaton reads the input from left to right and changes its state according to the word or symbol it reads. If the input follows a valid path and reaches a final state, the expression is accepted.

The aim of this FSA is to recognize absolute date expressions. Absolute dates identify a fixed calendar date or a named calendar event. Examples include “March 15”, “the 22nd of November”, “December 25, 2026”, and “Christmas”. Relative expressions such as “yesterday” or “the day before yesterday” are not included.

Word-class shorthand

To avoid drawing a separate transition for every month, day, and holiday, groups of words are represented by shorthand labels.

MONTH = January, February, March, April, May, June, July, August, September, October, November, December.

DAY = a number from 1 to 31, such as 1, 2, 15, or 31.

ORDINAL = 1st, 2nd, 3rd, 4th, 5th, …, 31st.

FESTIVAL = Christmas, New Year’s Day, Valentine’s Day, Independence Day, and other named calendar holidays.

THE = the; OF = of; COMMA = ,; YEAR = a four-digit year such as 2025 or 2026.

FSA state transitions

The FSA has one start state and one accepting state. Three main paths are used to recognize the three common forms of absolute dates.

```
[START] -- MONTH -- DAY ----------------------> [FINAL]
   |
   +-- THE -- ORDINAL -- OF -- MONTH ----------> [FINAL]
   |
   +-- FESTIVAL -------------------------------> [FINAL]

[FINAL] -- COMMA -- YEAR ---------------------> [FINAL]
```

The diagram uses labels such as MONTH and FESTIVAL instead of listing every possible word on a separate edge. The state marked FINAL is an accepting state. The optional comma and year transition allows dates such as “March 15, 2026”.

1. FSA – Explanation and Examples (Continued)

Working of the FSA

Path 1: Month followed by day. The automaton starts at START. When it reads a month name, it follows the MONTH transition. It then reads a valid day number and reaches FINAL. Example: “March 15”.

Path 2: The followed by ordinal, of, and month. The automaton reads the word “the”, then an ordinal number such as “22nd”, then the word “of”, and finally a month name. Example: “the 22nd of November”.

Path 3: Named calendar event. If the input is a recognized holiday or named calendar event, the FESTIVAL transition takes the automaton directly to FINAL. Example: “Christmas”.

Optional year: Once a valid date has been recognized, the automaton may read a comma followed by a four-digit year. Example: “December 25, 2026”.

Accepted examples

• March 15 – accepted through MONTH → DAY.

• the 22nd of November – accepted through THE → ORDINAL → OF → MONTH.

• Christmas – accepted through FESTIVAL.

• December 25, 2026 – accepted through MONTH → DAY → COMMA → YEAR.

• New Year’s Day – accepted as a named calendar event if included in the FESTIVAL class.

Rejected examples

• yesterday – rejected because it is a relative date.

• the day before yesterday – rejected because it depends on the current day.

• March – rejected by this design because the month is not followed by a day.

• 32nd of November – rejected because 32 is not a valid day number.

Conclusion

The designed FSA recognizes common absolute date expressions using a small number of word classes and transitions. It is easy to understand and can be extended by adding more named holidays, date formats, or validation rules. A practical implementation should also check the number of days in each month and leap-year rules.

2. Regular Expressions for the Given Languages

The following expressions use Perl-compatible regular expression (PCRE) notation. In these expressions, ^ represents the beginning of a string or line, $ represents the end, + means one or more occurrences, * means zero or more occurrences, and \b represents a word boundary. A word is treated as an alphabetic sequence separated by whitespace or punctuation.

a. The set of all alphabetic strings

```
^[A-Za-z]+$
```

Explanation: [A-Za-z] matches any uppercase or lowercase English letter. The + symbol requires at least one letter. The ^ and $ symbols ensure that the entire string contains only alphabetic characters.

Examples accepted: Hello, NLP, Computer. Examples rejected: Hello123, data-science, 123.

b. Lowercase alphabetic strings ending in b

```
^[a-z]*b$
```

Explanation: [a-z] matches lowercase letters. The * symbol allows zero or more lowercase letters before the final b. The expression ends with b, so every accepted string must have b as its last character.

Examples accepted: b, ab, club, web. Examples rejected: B, Ab, book.

c. Two consecutive repeated words

```
\b([A-Za-z]+)\s+\1\b
```

Explanation: The first word is captured by ([A-Za-z]+). The \s+ part requires one or more spaces between the words. The backreference \1 requires the next word to be exactly the same as the first word. Word boundaries prevent matching only part of a longer word.

Examples accepted: “the the”, “Humbert Humbert”. Examples rejected: “the bug”, “the big bug”, “book books”.

d. Strings over the alphabet {a, b} where every a is immediately preceded and followed by b

```
^(?:b|bab)*b?$
```

Explanation: The alphabet is restricted to a and b. Every occurrence of a must appear inside the block “bab”, so it has b immediately before and after it. The block b allows additional b characters. Repeating these blocks allows valid strings of different lengths.

Examples accepted: b, bb, bab, babb, bbab, babbab. Examples rejected: a, ab, ba, baa, aba.

The empty string is also accepted by this expression because the repeated group and optional final b can both occur zero times.

2. Regular Expressions (Continued)

e. Lines that start with an integer and end with a word

```
^[0-9]+.*\b[A-Za-z]+\s*[.!?]?$
```

Explanation: ^ ensures that matching begins at the start of the line. [0-9]+ matches an integer containing one or more digits. .* allows any content in the middle. The final alphabetic word is matched by \b[A-Za-z]+, and optional spaces and sentence punctuation may follow it.

Example accepted: “25 students attended the class.” Example rejected: “Students attended class 25.”

f. Strings containing both the complete words grotto and raven

```
^(?=.*\bgrotto\b)(?=.*\braven\b).*$
```

Explanation: The first positive lookahead checks that the complete word “grotto” occurs somewhere in the string. The second positive lookahead checks for the complete word “raven”. The remaining part matches the full string. Word boundaries ensure that “grottos” does not count as the word “grotto”.

Examples accepted: “The grotto contains a raven.”, “raven near the grotto”. Examples rejected: “The grottos are dark.”, “The raven is here.”

g. Pattern that places the first word of an English sentence in a register

```
(?m)(?:^|[.!?]\s+)[\"'(\[]*([A-Za-z]+)
```

Explanation: The pattern looks for the beginning of a line or the position after sentence-ending punctuation such as a full stop, question mark, or exclamation mark. Optional opening quotation marks or brackets are skipped. The first alphabetic word is captured in group 1.

In Perl-style notation, the captured word can be accessed using $1 after a successful match. In Python, it can be accessed using match.group(1). The (?m) option enables multiline mode so that ^ can match the beginning of each line.

Example: For the text “Hello, world. How are you? Fine!”, the captured first words are “Hello”, “How”, and “Fine”.

Summary of regular expression symbols

^ – beginning of string or line; $ – end of string or line; [A-Za-z] – alphabetic character; [0-9] – digit; + – one or more; * – zero or more; ? – optional; \s – whitespace; \b – word boundary; ( ) – capturing group; \1 – backreference to the first captured group.

Conclusion: Regular expressions provide a compact way to describe and recognize text patterns. They are useful for validating strings, finding repeated words, and extracting words from sentences.

3. Minimum Edit Distance: Compare “drive” with “brief” and “divers”

Minimum edit distance is the minimum number of editing operations needed to convert one word into another. It is commonly used in spelling correction, text comparison, and natural language processing.

Edit operations used

In this answer, Levenshtein distance is used. Each operation has a cost of 1.

1. Insertion: Add one character to the word.

2. Deletion: Remove one character from the word.

3. Substitution: Replace one character with another character.

A. Find the distance between “drive” and “brief”

One valid sequence of operations is:

Step 1: drive → brive. Substitute d with b. Cost = 1.

Step 2: brive → briee. Substitute v with e. Cost = 1.

Step 3: briee → brief. Substitute the final e with f. Cost = 1.

Total cost = 1 + 1 + 1 = 3.

Therefore, the edit distance between “drive” and “brief” is 3.

B. Find the distance between “drive” and “divers”

One valid sequence of operations is:

Step 1: drive → dive. Delete the character r. Cost = 1.

Step 2: dive → diver. Insert r at the end. Cost = 1.

Step 3: diver → divers. Insert s at the end. Cost = 1.

Total cost = 1 + 1 + 1 = 3.

Therefore, the edit distance between “drive” and “divers” is 3.

C. Comparison

Distance(drive, brief) = 3.

Distance(drive, divers) = 3.

Both words require three editing operations when converting from “drive”. Therefore, neither “brief” nor “divers” is closer to “drive” under the Levenshtein distance with equal insertion, deletion, and substitution costs.

Final conclusion

The minimum edit distance calculation shows that “drive” is equally close to “brief” and “divers”. Both comparisons have a minimum edit distance of 3. This demonstrates how edit distance can be used to compare words based on the number of character-level changes required.

Result: The FSA for absolute dates was designed, regular expressions for all seven languages were written, and minimum edit distances were calculated successfully.